

TECNOLOGÍAS DE LA INFORMACIÓN EN LINEA INGENIERÍA DE SOFTWARE

3 créditos



Profesor Autor:

Ing. Freddy Malla García, Mtr.

Titulaciones	Semestre
INGENIERÍA DE SOFTWARE	SEXTO

Tutorías: El profesor asignado publicará en el entorno virtual de aprendizaje online.utm.edu.ec, y sus horarios de conferencias se indicarán en la sección **CRONOGRAMA**.

PERIODO ABRIL - AGOSTO 2026



Contenido

.....	iii
Resultado de aprendizaje de la asignatura.....	iii
Unidad I Fundamentos de la Ingeniería de Software	1
1. Consideraciones generales.	1
1.1. Proceso de software.....	6
1.1.1. Actividades fundamentales de un proceso de software	6
1.1.2. Que incluye el proceso de software.....	7
1.2. Metodologías de software.	8
1.2.1. Metodologías de desarrollo y su relación con los procesos y procedimientos	8
1.2.2. Metodología – elementos	9
2. Modelos de ciclo de vida.	10
2.1. Modelo general de proceso.....	11
2.1.1. Actividades estructurales del proceso.....	13
2.2. Modelos de proceso prescriptivos	17
2.2.1. Modelo de la cascada	17
2.2.2. Modelos de proceso incremental.....	21
2.2.3. Modelos de procesos evolutivo	23
2.2.4. Modelo espiral	24
2.2.5. Modelos Concurrentes	27
2.3. Modelos proceso especializado.....	28
2.3.1. Desarrollo basado en componentes	28
2.3.2. El modelo de métodos formales	29
2.3.3. Desarrollo de software orientado a aspectos	30

2.4. Modelo proceso unificado	31
2.4.1. Fases del proceso unificado	32
2.5. Modelo proceso personal y equipo	34
2.5.1. PPS Proceso personal del software	35
2.5.1.1. Planeación	35
2.5.1.2. Diseño de alto nivel	35
2.5.1.3. Revisión del diseño de alto nivel	36
2.5.1.4. Desarrollo	36
2.5.1.5. Post Mortem	36
2.5.2. Proceso del equipo de software (PES)	37
Referencia Bibliográfica	39

Índice de Figuras

Figura 1 Capas de la ingeniería de software	5
Figura 2 Estructura de un proceso del software	12
Figura 3 Flujo de Proceso	16
Figura 4 Modelo de Cascada	18
Figura 5 Modleo en V	19
Figura 6 Modelo Incremental	22
Figura 7 Proceso Incremental	22
Figura 8 Modelo de espiral	26
Figura 9 Proceso en Espiral	26
Figura 10 Un elemento del modelo de proceso concurrente	27
Figura 11 Proceso Unificado	32

Índice de Tablas

Tabla 1 Actividades Sombrillas	14
Tabla 2 Etapas que incorpora el Desarrollo Basado en Componentes ..	28



Ingeniería de Software



Resultado de aprendizaje de la asignatura

Desarrollar en el estudiante habilidades cognitivas, metacognitivas, afectivas y sociales para que enfrenten con autonomía los procesos de auto aprendizaje para la aplicación de modelos, modelados y técnicas de la ingeniería de software en la elaboración de especificaciones de requerimientos y diseño de software.



Unidad 1: Fundamentos de Ingeniería de software

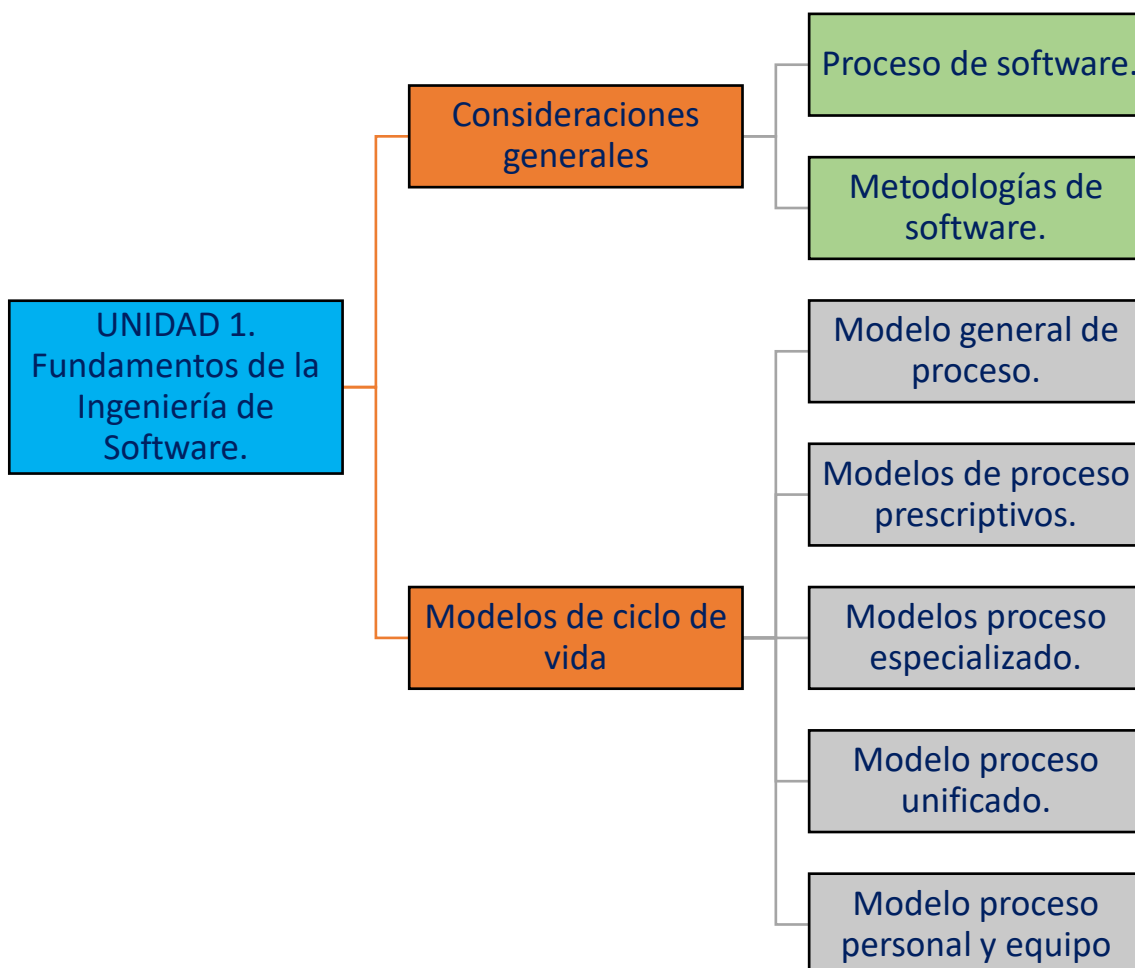
Resultado de aprendizaje de la unidad: Describir conocimientos sobre ingeniería del software, el proceso del software, los modelos de ciclo de vida y metodologías del software.



Introducción

Desarrollar en el estudiante habilidades cognitivas, metacognitivas, afectivas y sociales para que enfrenten con autonomía los procesos de auto aprendizaje para la aplicación de modelos, modelados y técnicas de la ingeniería de software en la elaboración de especificaciones de requerimientos y diseño de software. Describir conocimientos sobre ingeniería del software, el proceso del software, los modelos de ciclo de vida y metodologías del software, requerimientos del software (ERS), metodología de desarrollo ágil, diseño de arquitectura, diseño de componentes y diseño de interfaz de usuario.

Ejes temáticos



Organización de la lectura para el estudiante por semana del compendio

Semanas	Paginas
Semana 1	Página 2 -15
Semana 2	Página 16 - 30
Semana 3	Página 31 – 44

Unidad I Fundamentos de la Ingeniería de Software

1. Consideraciones generales.

¿Qué es Software?

El software de PC es el producto que construyen los desarrolladores expertos y al que posteriormente le brindan mantenimiento a lo largo de un periodo de tiempo incluyendo programas que se ejecutan en una PC de cualquier tamaño y arquitectura(Pressman, 2010).

La ingeniería de software se compone de un proceso, un conjunto de métodos (prácticos) y una gama de herramientas que permiten a los desarrolladores crear software informático de alta calidad(Pressman, 2010).

¿Quién lo hace?

Los expertos ingenieros desarrolladores de software crean y brindan mantenimiento al software, y prácticamente todo el mundo lo emplea ya sea en forma directa o indirecta (Pressman, 2010).

¿Por qué es importante?

El software tiene su importancia ya que se ha vuelto una parte imprescindible de nuestras vidas, y mucho más en el comercio, cultura y actividades cotidianas. La ingeniería de software es fundamental ya que posibilita crear sistemas complicados en un tiempo moderado y con alta calidad(Pressman, 2010).

¿Cuáles son los pasos?

El programa de PC se crea de igual forma que otro producto, con la aplicación de un proceso ágil y adaptable para lograr un resultado de gran calidad, de manera que satisfaga las necesidades de las personas que aprovecharán el producto(Pressman, 2010).

¿Cuál es el producto final?

A partir de la perspectiva de un ingeniero de programa, el producto terminado es el grupo de programas, contenido (datos) y otros productos completos que conforman el programa de PC(Pressman, 2010).

Desde la apreciación del usuario, el producto culminado es la información resultante de alguna manera transforma al mundo en el que vive (Pressman, 2010).

Dominios de aplicación del software

En la actualidad, existen siete grandes categorías de programas informáticos que presentan desafíos continuos para los ingenieros de programas, entre ellos tenemos: (Pressman, 2010)

1. Software de sistemas:

Grupos de programas escritos para servir a otros programas, ejemplo compiladores, editores y herramientas para administrar archivos, software de redes (García et al., 2019). El software de sistema procesa estructuras de información complicadas pero deterministas, sin embargo, otras aplicaciones de sistemas procesan datos indeterminados. Entonces, el área de software de sistemas se caracteriza por: alta interacción con el hardware de la computadora, uso intensivo de varios usuarios múltiples, operación concurrente que requiere secuenciación, recursos compartidos y administración de un proceso sofisticado, datos complejos y múltiples interfaces externas (Pressman, 2010).

2. Software de aplicación:

Programas aislados que solventan una necesidad específica de negocios. Los programas en esta área procesan información comerciales o técnicos en un modo que proporciona las operaciones de negocios y ayudan a la toma de resoluciones administrativas o normas (Pressman, 2010).

El software de aplicación no solo se usa para aplicaciones convencionales de procesamiento de datos sino también para controlar tareas de negocios en tiempo real (Pressman, 2010).

3. Software de ingeniería y ciencias: se ha caracterizado por algoritmos.

Las aplicaciones van desde la astronomía hasta la vulcanología, desde restricciones automotrices hasta la dinámica orbital del transbordador espacial, y desde la biología molecular hasta la fabricación automatizada. Sin

embargo, los algoritmos numéricos convencionales están siendo abandonados por las aplicaciones modernas dentro del área de la ingeniería y las ciencias(Pressman, 2010).

El diseño asistido por ordenadores, la simulación de sistemas y otros programas interactivas, comenzaron a realizarse en tiempo real considerando las funcionalidades del software de sistemas(Pressman, 2010).

4. Software incrustado:

Habita internamente en un producto o sistema y se emplea para implementar y supervisar características y funciones el beneficiario final y para el sistema en sí. El software incrustado realiza funciones limitadas y particulares como el control del tablero de un horno de microondas o proporciona un aporte importante de funcionamiento y control como funciones digitales en un automóvil, control del combustible, del tablero de control y de los sistemas de frenado(Pressman, 2010).

5. Software de línea de productos:

Esta creado para brindar una capacidad definida para uso de varios consumidores. El software de línea de productos se centra en algún mercado limitado y particular (por ejemplo, control del inventario de productos) o se dirige a mercados masivos de consumidores (procesamiento de textos, hojas de cálculo, gráficas por computadora, multimedios, entretenimiento, administración de base de datos y aplicaciones para finanzas personales o de negocios)(Pressman, 2010).

6. Aplicaciones web:

Comúnmente denominadas " webapps ", esta categoría de software centrado en la red, cubre una amplia gama de aplicaciones. Las webapps incorporan archivos de hipertexto que están vinculados brindando información con de texto y gráficos definidos. Desde el advenimiento de la Web 2.0, las aplicaciones web han avanzado hacia entornos informáticos complejos que brindan no solo funcionalidad discreta, funcionalidad computacional y contenido a los usuarios finales, sino también, se integra con bases de datos empresariales y aplicaciones de la industria(Pressman, 2010).

7. Software de inteligencia artificial:

Usa algoritmos no numéricos para solucionar Inconvenientes complejos que no pueden resolverse fácilmente mediante cálculo o análisis directos. Las aplicaciones en este campo incorporan robótica, sistemas expertos e identificación de patrones (imágenes y sonidos), redes neuronales artificiales, representación de teorías y juegos(Pressman, 2010).

Computación en un mundo abierto: Con la vertiginosa evolución de las redes inalámbricas es posible que esto conduzca a una computación verdaderamente generalizada y distribuida. El desafío para los desarrolladores de software será crear sistemas y programas de aplicación que permitan a los dispositivos móviles, computadoras personales y sistemas comerciales comunicarse a través de enormes redes(Pressman, 2010).

Construcción de redes: la World Wide Web se está convirtiendo rápidamente en un poderoso centro informático y proveedor de contenido. El reto del desarrollador de software es diseñar arquitecturas simples (por ejemplo, planeación financiera personal y aplicaciones complejas que dan ventaja en el mercado dirigido a usuarios finales en todo el mundo)(Pressman, 2010).

Fuente abierta: tendencia creciente que da como resultado la distribución de código fuente para aplicaciones de sistemas (por ejemplo, sistemas operativos, bases de datos y ambientes de desarrollo) de modo que mucha gente pueda apoyar a su mejora. El reto de los expertos de software es procesar código fuente que sea auto descriptivo, y desarrollar tecnologías que otorgarán tanto a los usuarios como a los desarrolladores conocer los cambios hechos y cómo se manifiestan dentro del software(Pressman, 2010).

Definiciones de ingeniería de software

La ingeniería de software es la creación y el uso de los fundamentos de la ingeniería de software, con el propósito de desarrollar software económicamente confiable y Funciona de manera efectiva en una máquina real (García et al., 2019).

La ingeniería de software es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento de software (García et al., 2019).

Fundamento en el que se apoya la IS:

Compromiso con la CALIDAD

Capas de la Ingeniería de Software:



Figura 1 Capas de la ingeniería de software

Fuente:(Pressman, 2010)

La ingeniería de software es una tecnología con varias capas, debe basarse en un **compromiso organizacional con la calidad** (García et al., 2019).

Proceso: estructura que debe crearse para tener una eficaz de tecnología de ingeniería de software, es una serie de acciones que conducen a un fin (García et al., 2019).

Proceso de software que constituye la base para gestionar proyectos de software y crear el contexto en el que se emplean los métodos de ingeniería y productos de trabajo creados (formularios, documentos, datos, informes, plantillas, etc.). Se establecen estándares, se asegura la calidad y se gestionar el cambio adecuadamente(García et al., 2019).

Métodos: Aportan experiencia técnica para el desarrollo de software. Integran muchos tipos de tareas como comunicación, análisis de requisitos y modelado de diseño, construcción, prueba y soporte del programa. Los métodos están basados en un grupo de principios esenciales que rigen cada área de la tecnología, encierran actividades de modelación de procesos y otras técnicas descriptivas(García et al., 2019).

Herramientas: facilita ayuda automática o semiautomática para los procesos y métodos. Cuando se integran herramientas para informar puede ser utilizado por otra persona, queda determinado un sistema llamado Ingeniería de software acudido por computadora (García et al., 2019).

1.1. Proceso de software.

Un **proceso de software** es una fusión de **actividades, acciones y tareas** ejecutadas al crearse un producto de trabajo (Pressman, 2010).

Una **actividad** opta por conseguir un objetivo amplio y desarrollarse independientemente del alcance y la escala del proyecto, es la complejidad del esfuerzo, o la precisión con la que se utilizará la tecnología (Pressman, 2010).

Una **acción** (diseño arquitectónico) incorpora tareas que conducen a la creación de un producto de trabajo principal (por ejemplo, un modelo de diseño arquitectónico) (Pressman, 2010).

Una **tarea** se enfoca en un propósito bien definido (por ejemplo, hacer una prueba unitaria) que resultan tangible (Pressman, 2010).

En ingeniería de software, el proceso no es una regla estricta de cómo desarrollar de programas informáticos, más bien, es una visión adaptativa que permite los individuos que realizan el trabajo (equipo del programa) investiguen y seleccionan el grupo de procedimientos y tareas para el trabajo (Pressman, 2010).

El objetivo siempre es entregar los programas a tiempo y de suficiente calidad para compensar a los que patrocinaron su creación y quienes la usarán (Pressman, 2010).

1.1.1. Actividades fundamentales de un proceso de software

Un proceso de software es una serie de actividades y resultados relacionados que producen un producto del programa, estas operaciones son realizadas por ingenieros de software. Existen diversos procesos de software, pero todos deben integrar estas cuatro actividades de proceso básicas fundamentales para la ingeniería del software (Aguilar Vera et al., 2017).

La especificación del software: aquí se debe definir el software a producir con sus respectivas restricciones sobre su operación (quienes lo definen son el cliente y el ingeniero) (Aguilar Vera et al., 2017) (JAN Sommerville, 2005).

Desarrollo de software (diseño e implementación): se diseña y programa cumpliendo con las especificaciones(Aguilar Vera et al., 2017) (JAN Sommerville, 2005).

Validación de software: se valida el software para garantizar que coincida con lo que requiere el cliente(Aguilar Vera et al., 2017) (JAN Sommerville, 2005).

Evolución del software: donde se realizan cambios al software para acoplarlo y satisfacer las necesidad del cliente y el mercado(Aguilar Vera et al., 2017) (JAN Sommerville, 2005).

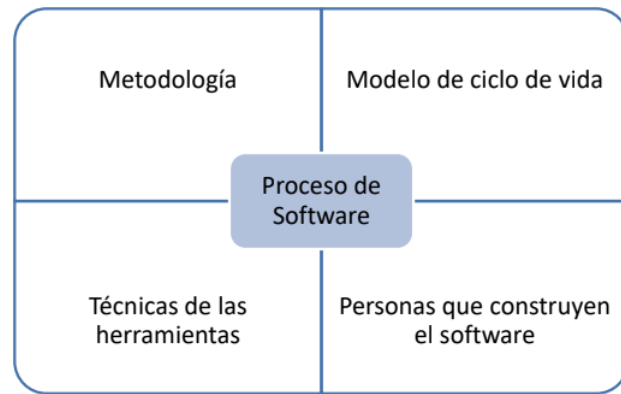
La estructura del proceso construye el fundamento para el proceso completo de la ingeniería de software definiendo un pequeño número de operaciones estructurales aplicadas a todos los proyectos de software independientemente de su tamaño o complejidad(Pressman, 2010).

Además, la arquitectura del proceso contiene operaciones que se emplean a todo el proceso del software. La estructura de proceso común de ingeniería de software consta de cinco actividades(Pressman, 2010).

1.1.2. Que incluye el proceso de software

El proceso de desarrollo de software puede incluir:

- **Modelo de ciclo de vida** en donde divide el desarrollo en etapas y describa las actividades paso a paso que debe hacerse, proporciona criterios para establecer cuándo cada etapa desarrollo está completo y e identificación de productos/artefactos/salidas para cada etapa(Pickin & Valls, 2006).
- **Herramientas y equipos de auditoría.**
- **Cuidar de sus empleados y organización**
- **Restricciones en actividades y artefactos, herramientas, personal, etc** (Pickin & Valls, 2006).



• Para muchos autores: proceso de desarrollo de software = ciclo de vida de software

1.2. Metodologías de software.

1) Conjunto de pasos y procedimientos que deben seguirse para el desarrollo de software.

2) Conjunto de filosofías, fases, procedimientos, reglas, técnicas, herramientas, documentación y aspectos de formación para los desarrolladores de SI.

3) Conjunto de procedimientos, técnicas, herramientas y soporte documental que ayuda a los desarrolladores a realizar nuevo software.

1.2.1. Metodologías de desarrollo y su relación con los procesos y procedimientos

Todo el desarrollo del software se puede caracterizar como un bucle de resolución de problemas en el que se encuentran cuatro etapas de solución:

- Estado actual (representa el estado actual del problema)
- Definición de problema
- Desarrollo técnico
- Integración de solución

1.2.2. Metodología – elementos

Elementos	Descripción
Técnicas	- Se utilizan para aplicar un Procedimiento - Pueden ser Gráficas y/o Textuales - Determinan el formato de los Productos resultantes en cada Tarea
Actividades y Tareas	- El Proceso se descompone hasta el nivel de - Actividades y Tareas (actividades elementales)
Productos	- Obtenidos como resultado de seguir un - Procedimiento - Pueden ser Intermedios o Finales
Herramientas Software	Proporcionan soporte a la aplicación de las Técnicas
Procedimientos	- Definen la forma de llevar a cabo las Tareas - Vínculo de Comunicación entre Usuarios y desarrolladores



2. Modelos de ciclo de vida.

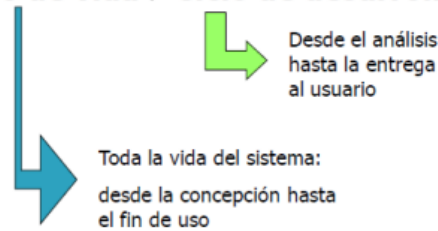
¿Qué es ciclo de vida de un software?

El ciclo de vida de desarrollo de software (conocido como SDLC o ciclo de vida de desarrollo de sistemas-*Systems Development Life Cycle*) proporciona las etapas esenciales para verificar el desarrollo de software, asegurando que cumpla con los requisitos de la aplicación y comprobar los procedimientos de desarrollo, asegurando que los métodos empleados sean adecuados, comienza cuando se inventa un software y concluye cuando el producto ya no está disponible para su uso.(*Ciclo de Vida Del Software: Todo Lo Que Necesitas Saber*, 2020).

Su origen reside en que es muy costoso corregir problemas que se descubran tarde en la fase de implementación; aplicando las metodologías correctas, se puede detectar en el momento adecuado para que los desarrolladores puedan concentrarse en la calidad del software, la puntualidad y los costos asociados(*Ciclo de Vida Del Software: Todo Lo Que Necesitas Saber*, 2020).

Es el proceso que se sigue para construir, entregar y hacer evolucionar el software, desde la concepción de una idea hasta la entrega y el retiro del sistema. Representa todas las actividades y artefactos (productos intermedios) necesarios para desarrollar una aplicación. Existen diferentes ciclos de desarrollo de software, el estándar ISO/IEC/IEEE 12207:2017 define: “Un marco común para los procesos del ciclo de vida del software, en términos bien definidos, al que puede referirse la industria del software. Incluye los procesos, actividades y deberes que se aplican al adquirir, proporcionar, desarrollar, operar, mantener o disponer de sistemas, productos y servicios de tecnología de la información. Estos procesos del ciclo de vida se llevan a cabo a través de la participación de los grupos de interés, con el objetivo final de lograr la satisfacción del cliente(ISO, 2017).

Ciclo de vida ≠ Ciclo de desarrollo



Las principales etapas que forman el ciclo de vida de desarrollo de software son:

Modelos de ciclo de vida

Un modelo de ciclo de vida es un modelo abstracto especial que representa el ciclo de vida de un programa. El modelo de ciclo de vida a menudo se denomina ciclo de vida de desarrollo de software. (SDLC, siglas inglesas). IEEE Standard Glossary of Soft. Eng. Terminology(Pickin & Valls, 2006)

2.1.Modelo general de proceso

El proceso se reduce como un grupo de actividades, acciones y tareas que se ejecutan cuando se necesita crear un producto final. Cada actividad, acción y tarea se encuentra en una estructura o patrón que define su relación con el proceso como entre sí(Pressman, 2010).

Un modelo de proceso de software es una representación simplificada de un proceso de software. Estos modelos pueden agregar actividades como parte de los productos y procesos de software y los roles de los involucrados en la ingeniería de software. Algunos ejemplos de este tipo de modelos son:(JAN Sommerville, 2005)

Modelo de flujo de trabajo: presenta la sucesión de actividades en el proceso junto ligado a sus entradas, salidas y dependencias. Las actividades en este modelo representan acciones humanas(JAN Sommerville, 2005).

Un modelo de flujo de datos o de actividad: mostrar todo el proceso actividades, cada uno de ellos efectúa una transformación sobre los datos. la

entrada al proceso la muestra como una especificación y se convierte en salida como un diseño. Pueden representar las transformaciones a realizar por humanos o por computadora(JAN Sommerville, 2005).

Modelo de rol/acción (animado). Incorpora los roles de las individuos implicadas en el proceso del programa y las actividades de las que son responsables(JAN Sommerville, 2005).

En la siguiente figura el proceso de software se muestra en forma de diagrama esquemático, en la que cada actividad está compuesta por una fusión de acciones de ingeniería de software y cada uno de estos, está definido por un grupo específico de tareas a realizar, los productos del trabajo a producir, aseguramiento de la calidad que se requieren y las referencias necesarias para evaluar el progreso(Pressman, 2010).

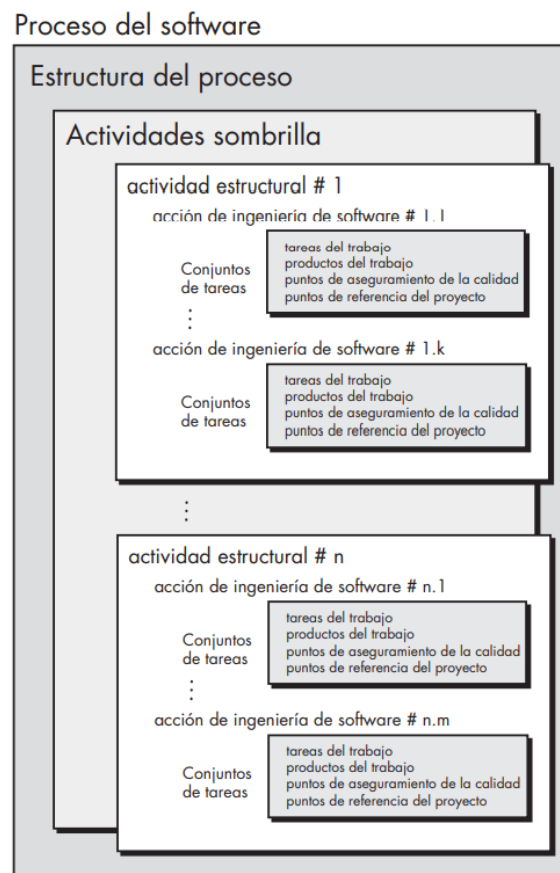


Figura 2 Estructura de un proceso del software

Fuente:(Pressman, 2010)

La ingeniería de software define cinco actividades estructurales: comunicación, planeación, modelado, construcción y despliegue(Pressman, 2010).

2.1.1.Actividades estructurales del proceso

Una estructura general para la ingeniería de software define cinco actividades estructurales: comunicación, planeación, modelado, construcción y despliegue.

Comunicación. Es importante comunicarse y colaborar con los clientes antes de que comience cualquier trabajo de ingeniería (y otras partes interesadas). Busca comprender los objetivos de los cómplices para el proyecto y recopilación de requisitos necesario que permitan definir las características y la funcionalidad del software(Pressman, 2010).

Planeación. crea un mapa que guía al equipo, con actividades desafiantes, este mapa es llamado plan del proyecto del software, especifica el trabajo de ingeniería de software detallando las tareas técnicas que se realizarán detectando los riesgos potenciales, los recursos necesarios, el producto del trabajo que se completará y el cronograma del proyecto(García et al., 2019).

Modelado. Si usted es el diseñador, constructor de puentes, ingeniero aeronáutico, carpintero o arquitecto o trabaja con modelos todos los días. Crea un "esbozo" del objeto que ayude a comprender: cómo se vería arquitectónicamente y cómo se juntan los componentes básicos y muchas otras funciones. Si es necesario, mejore el diagrama con más datos para entender mejor el problema y cómo solucionarlo. Un experto de software crea lo mismo elaborando modelos para concebir mejor los requisitos de software y diseño que los satisfarán(Pressman, 2010).

Construcción. Esta actividad fusiona la creación de código (manual o automáticamente) con las pruebas esenciales para encontrar los errores que contiene(Pressman, 2010).

Despliegue. Software (total o parcialmente culminado) que es evaluado y retroalimentado por el consumidor que lo revisa(Pressman, 2010).

Estas cinco actividades se utilizan en el desarrollo de programas pequeños y simples, en la construcción de grandes aplicaciones web y en la ingeniería de sistemas.

Detalles de procesos de software de computadora grandes y complejos varía en cada caso, pero la estructura de los procesos es la misma. Para muchos proyectos de software, las actividades se emplean repetidamente según avanza el proyecto. En otras palabras, la comunicación, la planeación, el modelado, la construcción y el despliegue se realizan mediante una serie de iteraciones del proyecto. Cada iteración crea un incremento del programa adicional para los participantes con características generales y la funcionalidad del programa, conforme sucede cada aumento, el programa se vuelve más perfecto (Pressman, 2010).

Las actividades estructurales del proceso de ingeniería de software se integran con actividades sombrilla, estas se emplean en todo el tiempo de un proyecto de software y brindan soporte al equipo que construye, administrar y monitorear el progreso, calidad, cambio y riesgos. Las actividades sombrillas suelen ser:

Tabla 1 Actividades Sombrillas

Actividades Sombrillas	Descripción
Seguimiento y control del proyecto de software:	El equipo de software evalúa el progreso diferenciándolo del plan del proyecto acogiendo las acciones necesarias para cumplir con el cronograma de actividades.
Administración de riesgos:	Valorar los peligros que pueden influir los resultados del proyecto o la calidad del producto.
Aseguramiento de la calidad del software:	Identificar e implementar las actividades necesarias para avalar la calidad del programa.
Revisiones técnicas:	Valoración de productos de trabajo de ingeniería de software para detectar y descartar fallos antes de que se avance a la otra actividad.
Medición:	Específica y recopila métricas de procesos, proyectos y productos para que el grupo brinde programas que satisfacen las necesidades de los

	participantes; es posible utilizarse con todas las demás actividades estructurales y sombrillas.
Administración de la configuración del software:	Gestiona los efectos de los cambios durante todo el proceso del programa.
Administración de la reutilización:	Estipula criterios de reutilización del producto del trabajar (incluidos componentes de software) y crea mecanismos para conseguir componentes reutilizables.
Preparación y producción del producto del trabajo:	Junta actividades necesarias para efectuar productos de trabajo, como formularios, documentos, plantillas y lista.

Fuente: (Pressman, 2010)

En la figura 3 Flujo del proceso, se describe cómo se organiza secuencialmente y en el tiempo la estructura de las actividades, procedimientos y tareas que se dan en cada actividad. El proceso lineal lleva a cabo cada una de las cinco actividades estructuradas en secuencia comenzando con la comunicación y terminando con la ejecución (ver Fig. a). El flujo de proceso iterativo es repetir una o más actividades antes de pasar a la siguiente (ver Fig. b). Un flujo de proceso evolutivo escalable que ejecuta operaciones en un "bucle"(circular). Porque de cinco operaciones, cada episodio lleva a una versión más completa del programa (ver Fig. c). Un flujo de proceso paralelo (ver Fig. d) que realiza una o más actividades en paralelo con las demás (por ejemplo, se puede realizar el modelado de un aspecto del programa junto con la construcción de otro aspecto del software)(Pressman, 2010).

Flujo de Proceso

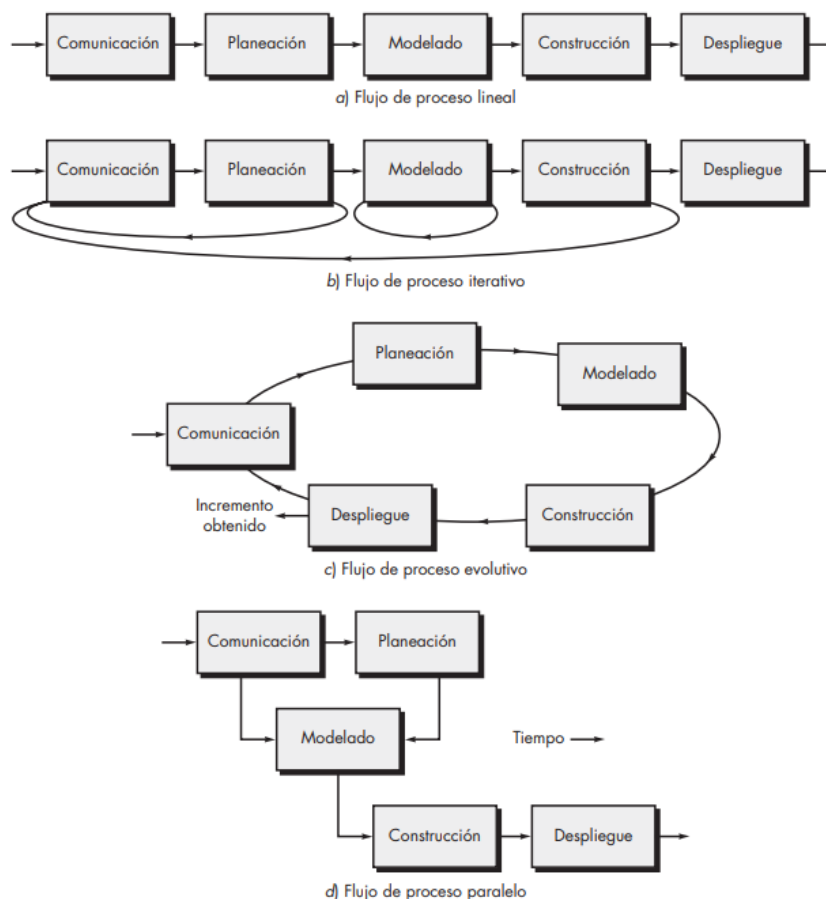


Figura 3 Flujo de Proceso
 Fuente: (Pressman, 2010)

Definición de actividad estructural

Aunque el primer capítulo describe cinco actividades estructurales y da definiciones para todos, el equipo de software necesitará mucha información antes de poder ejecutar exactamente uno de ellos como parte de un proceso programático. Surge así una cuestión importante: ¿qué medidas son adecuadas para una actividad estructural, teniendo la naturaleza del problema a resolver, las características de las personas que realizan el trabajo y los involucrados que financian el proyecto? (Pressman, 2010).

Para un pequeño proyecto de software requerido por una persona (lejos) con requisitos simples y directos, las actividades de comunicación pueden no incluir nada más que una simple llamada telefónica con los participantes adecuados. Por lo tanto, el único procedimiento necesario es una

conversación telefónica y tareas de trabajo (todas las tareas), que integran lo siguiente:(Pressman, 2010)

- Póngase en contacto con los participantes por teléfono.
- Análisis de necesidades y registrar notas.
- Organice las notas escritas en una breve declaración de requisitos.
- Emitir vía correo electrónico para su revisión y aprobación.

Si el proyecto es significativamente más complejo, con muchos participantes y todos ellos con un conjunto diferente (a veces contradictorio) de requisitos y actividades de comunicación puede incluir seis procedimientos diferentes (concepción, indagación, elaboración, negociación, especificación y validación. Cada uno de estos procedimientos técnicos tendrá muchas tareas de trabajo y una gran cantidad de productos finales diferentes(Pressman, 2010).

2.2.Modelos de proceso prescriptivos

Originalmente se propusieron modelos descriptivos de procesos para ordenar los enredos de desarrollo de software. Los modelos de procesos de software pueden contener las cinco actividades estructurales, pero cada uno ubica un énfasis diferente en él y define el proceso que lo invoca a cada actividad estructural de manera diferente(García et al., 2019).

2.2.1.Modelo de la cascada

Surgió en el año 1970 (Winston W. Royce), también conocido como *Ciclo de vida clásico o Lineal Secuencial*, debemos tener presente que antes de programar primero se debe diseñar y probar el programa antes de construirlo y ponerlo en operación(García et al., 2019).

Ventajas

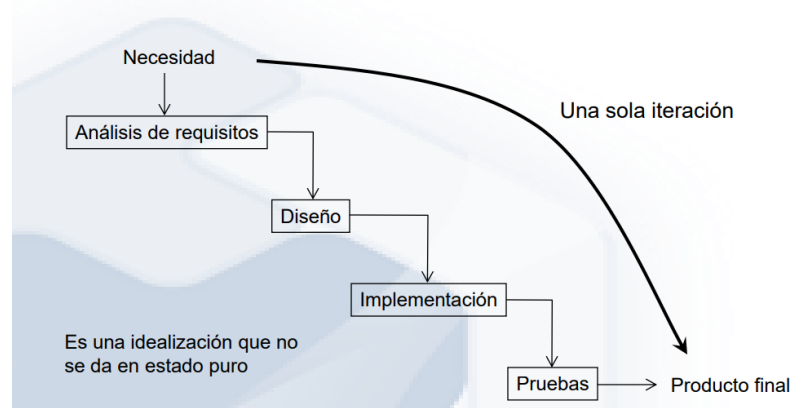
- Modelo y planificación simples y fáciles ellos.
- Sus etapas son entendidas por los desarrolladores.
- Los usuarios pueden entenderlo fácilmente
- Ayuda a atrapar problemas a bajo costo(García et al., 2019)

Desventajas

- Alto riesgo en nuevos sistemas por problemas de especificación y en el diseño.
- Es difícil para el cliente presentar todos los requisitos claramente al principio.
- Los clientes deben tener paciencia porque recibirán el producto al final (García et al., 2019).

Los proyectos reales rara vez siguen el proceso secuencial sugerido por el modelo, a pesar de que el modelo lineal admite repeticiones, y lo hace indirectamente. Como resultado, los cambios causan confusión a medida que avanza el equipo del proyecto (García et al., 2019).

Ejemplo: Proceso en Cascada



Fuente; (Braude, 2001)

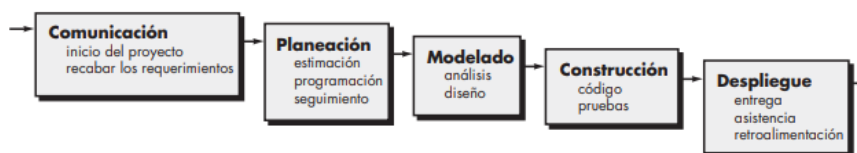


Figura 4 Modelo de Cascada

Fuente: (Pressman, 2010)

El modelo en cascada, propone un enfoque sistemático y secuencialmente para el desarrollo de software, comenzando con la especificación de los requisitos del cliente, a través de la planificación, el modelado, la construcción,

despliegue y terminando con el soporte completo del software (Fig. 1). La representación variante del modelo de cascada se conoce como modelo en V(Pressman, 2010).

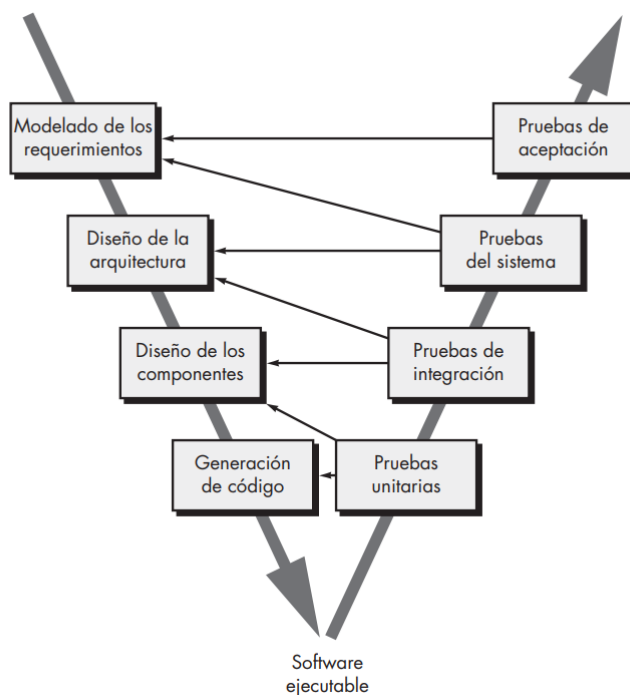


Figura 5 Modelo en V
Fuente: (Pressman, 2010)

La Fig. 2 muestra el modelo en V, donde la relación entre los procedimientos donde garantiza la calidad y lo relacionado con comunicación, modelado y construcción temprana(Pressman, 2010).

Cuando el grupo de software avanza para abajo desde el lado izquierdo de la V, los requisitos previos del problema van mejorando hacia la representación técnicas con cada uno de ellos con información más detallada sobre el problema y su solución. Después de generar el código, el grupo se mueve a la derecha de V, y básicamente realiza una serie de pruebas (acciones para garantizar la calidad, que valida cada modelo que se genera durante la transición del equipo cuando se dirigía hacia abajo a la izquierda, de hecho, no hay una diferencia fundamental entre el ciclo de vida clásico y modelo V. El modelo en V le ayuda a visualizar la situación de la aplicación desde los procedimientos de verificación y validación del trabajo de ingeniería inicial(Pressman, 2010).

El modelo de cascada es el modelo más antiguo en ingeniería de software. Sin embargo, durante las últimas tres décadas, las críticas a este modelo han llevado a que los defensores más obstinados cuestionen su eficacia [Han95]. Entre los problemas que a veces surgen al implementar el modelo en cascada se encuentran:(Pressman, 2010)

1. Los proyectos reales rara vez siguen el proceso secuencial sugerido por el modelo. Aunque el modelo lineal acepta la recursividad, lo hace indirectamente. Así que, los cambios causan confusión a medida que avanza el equipo del proyecto(Pressman, 2010).

2. A menudo, es difícil para los clientes definir claramente todos los requisitos. El modelo de cascada debe implementarse y luchar por aceptar la incertidumbre natural que se presente al inicio de muchos proyectos(Pressman, 2010).

3. Los clientes deben ser pacientes. Una copia de trabajo no estará disponible, hasta que el proyecto esté bien optimizado. Sería un desastre detectar un error grande teniendo el programa en funcionamiento(Pressman, 2010).

En un interesante análisis de proyectos reales, Bradac [Bra94] localizó que naturaleza lineal del ciclo de vida clásico alcanza un "estado de bloqueo" donde algunos miembros del equipo del proyecto deben esperar a que otros completen las tareas pertinentes. De hecho, ¡el tiempo de espera excede el tiempo que dedicas a ser productivo! Los estados de bloqueo suelen ocurrir más al principio y al final de un proceso secuencial lineal. El trabajo de software actual es acelerado y sufre cambios interminables (en las características, funciones y contenido de información), el modelo de cascada generalmente no es adecuado para este tipo de labor. Sin embargo, vale como un modelo de proceso útil en situaciones donde los requisitos son fijos y el trabajo avanza linealmente hasta su finalización(Pressman, 2010).

2.2.2. Modelos de proceso incremental

Existen circunstancias en donde los requisitos iniciales del software son razonablemente bien definidos, pero el alcance general del esfuerzo de desarrollo hace que el proceso lineal sea imposible (Pressman, 2010).

Además, puede haber una necesidad urgente de introducir rápidamente ciertas funcionalidades limitadas del software a usuarios y lo aumenta en las entregas de versiones de software posteriores, en tales casos, se selecciona el modelo de proceso diseñado para producir el programa paso a paso (en incremento) (Pressman, 2010).

El modelo incremental integra los elementos de los procesos lineales y paralelos. Con referencia a la Fig. 3, el modelo incremental emplea secuencias lineales de forma escalonada a medida que avanza el cronograma de actividades. Toda la secuencia lineal produce un "incremento" del Software susceptibles de entregarse [McD93] similar al de un producto en un proceso evolutivo, por ejemplo, el software de procesamiento de textos se ha desarrollado con un modelo incremental, la cual puede proporcionar funciones básicas de gestión de archivos en el primer incremento, edición y producción de documentos; en segundo lugar, proporcionará herramientas de edición y producción de documentos más avanzadas; en la tercera parte habrá una separación entre palabras y modificación y revisión de la ortografía; el cuarto proporcionará capacidades avanzadas de formato de página. Cabe señalar que el flujo de proceso para cualquier aumento puede incluir prototipo (Pressman, 2010).

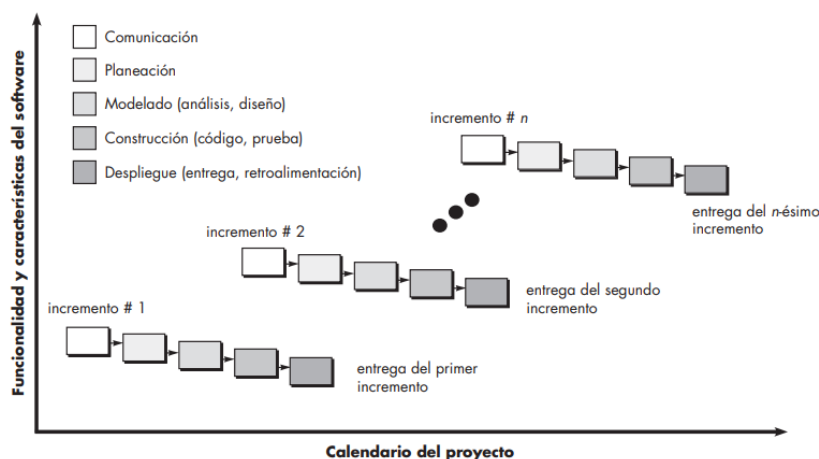


Figura 6 Modelo Incremental
Fuente: (Pressman, 2010)

Cuando se utiliza un modelo incremental, el primer valor agregado suele ser el producto fundamental. Esto significa que se abordan los requisitos previos, pero no se proporcionan algunas características suplementarias (conocidos, desconocidos). Los clientes utilizan el producto esencial (o según evaluación detallada). Después del uso y/o evaluación, se hace un plan para el siguiente paso. El plan contiene la modificación del producto necesario para satisfacer mejor las necesidades del cliente, así como para proporcionar características adicionales y de funcionalidad adicional y más. Este proceso se repite después de cada entrega inicial, hasta completar el producto final (Pressman, 2010).

Ejemplo: Proceso Iterativo e Incremental

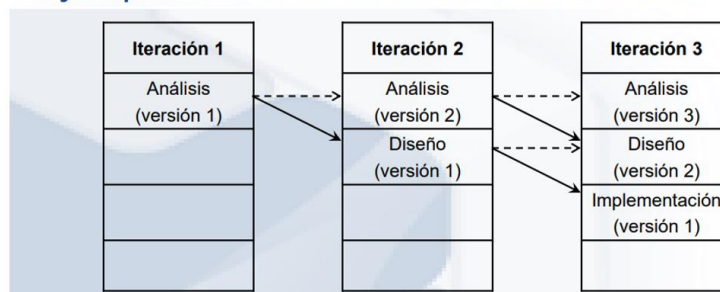


Figura 7 Proceso Incremental
Fuente: (Braude, 2001)

El modelo de proceso incremental se enfoca en entregar un producto funcional en cada incremento adicional. El primer incremento es la versión básica del producto final, pero proporcionan la capacidad de servir a los usuarios y también proporcionan una plataforma para la evaluación(Pressman, 2010).

El desarrollo incremental es particularmente útil cuando los empleados no están disponibles para implementar completamente el proyecto dentro del marco de tiempo establecido por la empresa. Los primeros incrementos se desarrollaron con pocos trabajadores. Si el producto básico es bien recibido, entonces se agrega más personal (si es necesario) para trabajar en el siguiente incremento. Hay un incremento planificado en la gestión de riesgos técnicos. Por ejemplo, un sistema grande tal vez requiera que se disponga de hardware nuevo donde se puede solicitar la disponibilidad y fecha de un nuevo material en desarrollo y cuya fecha de entrega sea incierta, en este caso, es posible planificar los primeros incrementos de forma que se evite el uso de estos dispositivos y así brindar una funcionalidad parcial para usuarios finales sin un retraso importante(Pressman, 2010).

2.2.3. Modelos de procesos evolutivo

El software, como todos los sistemas complicados, evoluciona con el tiempo. Los requisitos comerciales y de productos cambian a medida que avanza el desarrollo, ya que no es práctico trazar un camino directo al producto final; la falta de tiempo, las condiciones del mercado no pueden hacer un software perfecto, pero se debe lanzar una edición limitada para aliviar la presión competitiva o comercial; se comprende bien el conjunto de requerimientos o el producto básico, pero aún es necesario definir las extensiones del sistema. En estas y otras situaciones similares, es necesario un modelo de proceso que esté diseñado explícitamente para adaptarse a un producto que cambia con el tiempo(Pressman, 2010).

Los modelos evolutivos son iterativos. Destaca la forma en que permite desarrollar más y más versiones de software perfecto(Pressman, 2010).

2.2.4. Modelo espiral

Propuesto por Barry Boom (Boe88), el modelo espiral es un modelo sofisticado del proceso de software que va de la mano con la naturaleza iterativa de hacer prototipo con aspectos controlados y sistemáticos del modelo de cascada. Tiene el potencial para desarrollar rápidamente versiones cada vez más perfectas. Descripción de Boom [Boe01a] acerca del modelo:(Pressman, 2010)

El modelo de desarrollo en espiral es un modelador de procesos basado en el riesgo, utilizado para el enrutamiento de ingeniería concurrente con participantes múltiples de sistemas intensivo de software. Tiene dos características distintivas principales. Primero es un enfoque cíclico para aumentar gradualmente el grado de definición e implementación del sistema, al mismo tiempo que se reduce el nivel de riesgo en el mismo. El otro es un conjunto de puntos de referencia anclaje puntual para asegurar el compromiso de las partes con soluciones posibles y satisfactorias para ambas partes(Pressman, 2010).

Utilizando el modelo en espiral, el software se desarrolló en una serie de entregas evolutivas. En las primeras iteraciones, lo que se entrega puede ser un modelo o un prototipo, las iteraciones posteriores producen versiones más maduras del sistema diseñado.

Un modelo en espiral es dividido por el equipo de software en el equipo del programa descompone en un conjunto de actividades estructurales. Para fines ilustrativos, se utilizan las actividades estructurales generales ya analizadas. Cada uno de estos procesos representa una parte de la trayectoria espiral. Al principio del proceso de desarrollo, el equipo de software realiza actividades implícitas en donde encierra en un círculo en torno a la espiral en el sentido de las agujas del reloj, comenzando por el centro(Pressman, 2010).

Los riesgos se consideran a medida que avanza cada revolución, Se obtienen puntos de referencia puntuales en cada etapa de desarrollo mezclando productos de trabajo y condiciones existentes a lo largo de la trayectoria de la espiral(Pressman, 2010).

La primera ronda en torno a la espiral muestra el desarrollo de las especificaciones del producto; las siguientes vueltas sucesivas se aplican para desarrollar un prototipo y luego versiones de software cada vez más sofisticado. Cada vez que se pasa por el área de planeación, se modifica el plan del proyecto. Se ajustan costos y cronogramas de actividades basado en los comentarios obtenidos de los clientes después de la entrega. Además, el gerente del proyecto establece el número esperado de iteraciones requeridas para completar el programa. A diferencia de otros modelos de procesos que finalizan con la entrega del software, este modelo espiral se puede adaptar para aplicarse a lo largo de la vida del programa(Pressman, 2010).

El primer círculo alrededor de la espiral quizá represente un “proyecto de desarrollo del concepto” empezando en el centro de la espiral y continuar a través de las iteraciones Múltiplos hasta que se complete el desarrollo del concepto. Si el concepto se desarrolla en un producto real, el proceso continúa fuera de la espiral y comienza el “proyecto de desarrollo de nuevo producto”, los nuevos productos evolucionarán a través de cierto número de iteraciones alrededor de la espiral(Pressman, 2010).

Se puede usar un círculo alrededor de la espiral y representa un proyecto de mejora del producto, por lo tanto, permanece activo hasta que se elimine el programa. Hay veces que el proceso ha sido arreglado, pero cada vez que comience el cambio, en el punto de entrada apropiado (por ejemplo, mejoras de productos)(Pressman, 2010).

El modelo en espiral es un enfoque práctico para el desarrollo de sistemas y software a gran escala, con el desarrollo de software en curso, los desarrolladores y los clientes comprenden y reaccionan mejor a los riesgos en todos los niveles de desarrollo(Pressman, 2010).

El modelo espiral usa prototipos como un mecanismo para reducir el riesgo, y permite emplear la creación de prototipos en cualquier etapa del proceso de desarrollo del producto. Conserva el enfoque de escalón sistemático propuesto por el ciclo de vida clásico, pero lo incorpora a la estructura repetitiva donde refleja de manera más realista el mundo real. El modelo en

espiral requiere el estudio de los riesgos de ingeniería directamente en todas las etapas del proyecto y si se aplica correctamente, reducirá el riesgo antes de que se convierta en un problema(Pressman, 2010).

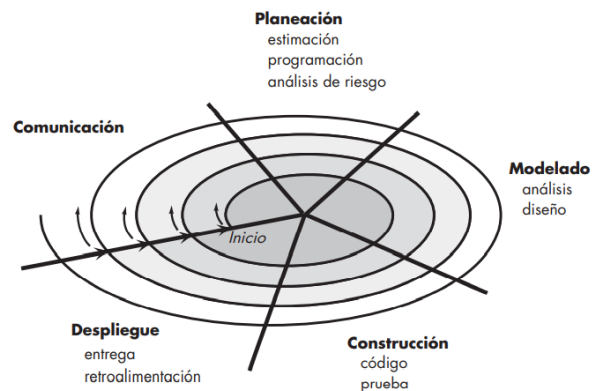


Figura 8 Modelo de espiral
Fuente: (Pressman, 2010)

Es difícil de convencer a clientes (particularmente en el caso de la subcontratación) para los que se puede gestionar el enfoque de extensión. Se requiere una experiencia significativa en evaluación de riesgos y se depende de ella para tener éxito. Sin duda habrá problemas si no se descubren y gestionan los riesgos significativos(Pressman, 2010)

Ejemplo: Proceso en Espiral

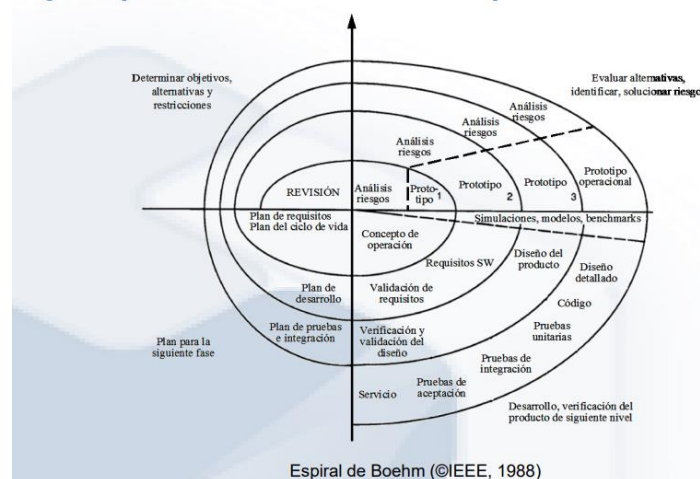


Figura 9 Proceso en Espiral
Fuente; (Braude, 2001)

2.2.5. Modelos Concurrentes

El modelo de desarrollo concurrente, a veces llamado ingeniería de concurrente, permite que un conjunto de programas represente los componentes iterativos y simultáneos de cualquiera de los modelos de procesos. Por ejemplo, las operaciones de modelado definida en el modelado espiral se logran llamando a una o más de las siguientes acciones de software: creación de prototipos, análisis y diseño(Pressman, 2010).

La siguiente figura 10 representación esquemática de una actividad de ingeniería de software en una actividad de modelado usando modelado concurrente(Pressman, 2010).

La actividad - modelado - puede estar en cualquiera de los estados mencionados en un momento dado. De manera similar, se puede representar otros procesos de manera similar, un trabajo o tarea (como comunicación o construcción), en donde un estado es algún modo de comportamiento observable externamente. Todas las actividades de ingeniería de software existen simultáneamente, pero en diferentes estados(Pressman, 2010).

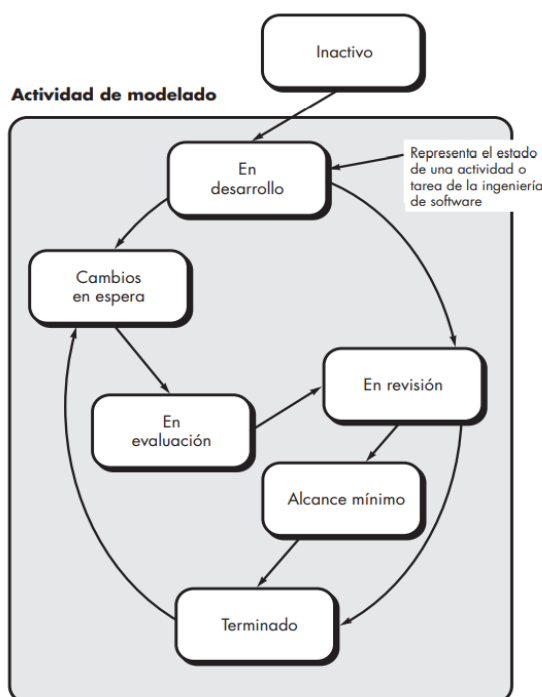


Figura 10 Un elemento del modelo de proceso concurrente
 Fuente: (Pressman, 2010)

2.3. Modelos proceso especializado

Modelos de procesos especializados que representan múltiples características de uno o más de los modelos convencionales. Sin embargo, estos modelos tienden a ser aplicables cuando se seleccionan enfoques de ingeniería de software especializados.

2.3.1. Desarrollo basado en componentes

Los componentes comerciales de software (COTS), desarrollados por los proveedores que los ofrecen como productos, ofrecen una operatividad que se persiguen con interfaces bien definidas que permiten integrar el componente en el software que se va a construir. Un modelo de desarrollo basado en componentes combina muchas de las características del modelo en espiral. Se desarrolla en la naturaleza evolutiva y requiere un enfoque iterativo para crear un programa. Sin embargo, el modelo de desarrollo basado en componentes crea aplicaciones a partir de segmentos de software prefabricadas (Pressman, 2010).

Las actividades de construcción y modelado comienzan con la identificación de candidatos de componentes. Pueden diseñarse como módulos o capas de software convencional o clases orientadas a objetos o paquetes de clases, independientemente de la tecnología utilizada en la construcción de componentes; el modelo de desarrollo basado en componentes incluye los siguientes pasos, implementan un enfoque evolutivo: (Pressman, 2010)

1. Se buscan y valoran, para el tipo de aplicación de que se trate, productos disponibles basados en componentes.
2. Se consideran los aspectos de integración de los componentes.
3. Se diseña una arquitectura del software para que adopte los componentes.
4. Se integran los componentes en la arquitectura.
5. Se generan pruebas exhaustivas para garantizar la funcionalidad apropiada

Tabla 2 Etapas que incorpora el Desarrollo Basado en Componentes

Fuente: (Pressman, 2010)

El modelo de desarrollo basado en componentes conduce a la reutilización del software y esto ofrece a los ingenieros de software una serie de ventajas en cuanto a la mensurabilidad. Si la reutilización de componentes forma parte de la cultura, el equipo de ingeniería de software puede reducir el tiempo del ciclo de desarrollo y los costos del proyecto (Pressman, 2010).

2.3.2. El modelo de métodos formales

Los modelos de métodos formales forman el conjunto de operaciones que conducen a la especificación matemática formal de programas informáticos. Los métodos formales le permiten definir, desarrollar y comprobar un sistema informático mediante el uso de notaciones matemáticas. Algunas organizaciones de desarrollo de software adoptan una variación de este enfoque llamada ingeniería de software de quirófano. Cuando se usan métodos formales durante el desarrollo, se encuentra un mecanismo para eliminar muchas dificultades que son arduas de superar usando otros paradigmas de la ingeniería de software. Lo que no está claro, incompleto e inconsistente será descubierto y corregido con más facilidad, no por una modificación especial, sino por la aplicación del análisis matemático. Si se aplican métodos formales utilizados en el proceso de diseño, serán la base de verificación del programa, y de esta forma permite detectar y corregir errores que pueden pasar desapercibidos. Aunque el modelo de métodos formales no es el más seguido, promete ofrecer software libre de defectos. Sin embargo, se han mencionado preocupaciones de su aplicabilidad en ambientes de trabajo:

- Desarrollar modelos formales requiere mucho tiempo y es costoso.
- Son pocos desarrolladores de software que tienen la formación necesaria para aplicar métodos formales, se requiere mucho entrenamiento.
- Es difícil aplicar modelos como mecanismos de comunicación para los clientes sin tecnología compleja.

A pesar de estas preocupaciones, el enfoque del método formal ha ganado popularidad entre los desarrolladores que necesitan crear software de seguridad de alta calidad (ejemplo, control electrónico de aeronaves y equipos

médicos) y entre los desarrolladores que sufrirían pérdidas financieras si algo sale mal con su software(Pressman, 2010).

2.3.3.Desarrollo de software orientado a aspectos

Independientemente del proceso del programa elegido, los creadores de programas complejos siempre implementan un conjunto localizado de características, funciones y contenido informativo. Las funciones localizadas del software están diseñadas como componentes (clases orientadas a objetos) se construyen como parte de la arquitectura sistemas. A medida que los sistemas informáticos modernos se vuelven cada vez más sofisticados (y complejos), existen ciertas preocupaciones (atributos o áreas de interés del cliente), inquietudes técnicas: se extienden a todas las arquitecturas. Algunos de ellos son propiedades de alto nivel de un sistema (ejemplo, seguridad y tolerancia a fallos). Otros afectan a la funcionalidad (reglas de negocio), otros son sistémicos (sincronización de tareas o gestión de memoria)(Pressman, 2010).

Cuando los intereses involucran múltiples funciones, características e información se las denomina preocupaciones globales. Los requerimientos del aspecto detallan preocupaciones globales influyentes a través de la estructura del sistema. El desarrollo de software orientado a aspectos (DSOA), también conocido como programación orientada a aspectos (POA), es un paradigma relativamente simple de ingeniería de software, que proporciona un proceso y un enfoque sistemático para definir, especificar, diseñar aspecto estructural: “mecanismos más allá de subrutinas y herencia para localizar la expresión de una preocupación global” [Elr01](Pressman, 2010).

Grundy [Gru02] discute otros temas en el contexto de lo que él llama ingeniería de componentes orientada a aspectos (ICOA) que utiliza los conceptos cortos a través de componentes de software separados verticalmente, conocidos como "aspectos", para describir las propiedades globales funcionales y no funcionales de los componentes. Los aspectos generales y del sistema incluyen la interfaz de usuario, la colaboración, la distribución, la constancia, la gestión de la memoria y el procesamiento de

transacciones, seguridad, integridad, etc. Los componentes pueden proporcionar o requerir uno o más "detalles de aspecto" relacionados con un aspecto específico, como el mecanismo de visualización, el alcance escalable y la capa de interfaz (lado de la interfaz de usuario); crear, transmitir y recibir eventos (lado de distribución); almacenamiento, recuperación e indexación de datos (aspectos de persistencia); autenticación, cifrado y acceso (lado de la seguridad); análisis transaccional, control de concurrencia y estrategia de registro (aspectos de las transacciones), entre otros. Cada detalle tiene una serie de propiedades relacionadas con características funcionales o no funcionales de detalles del aspecto (Pressman, 2010).

2.4. Modelo proceso unificado

En su libro fundamental, Unified Process, Ivar Jacobson, Grady Booch y James Rumbaugh [Jac99] estudian la necesidad de un proceso software "centrado y orientado a casos de uso, arquitectura, iterativa e incremental", con el siguiente: Actualmente, el programa está dirigido a sistemas más grandes y más complejos. Eso es por el hecho de que la computadora cada vez son más fuerte año tras año y los usuarios esperan más de ellos. Esta tendencia también está influenciada por el creciente uso de Internet para intercambiar todo tipo de información, la pasión por el software está creciendo, la evolución aumenta a medida que aprendemos entre un lanzamiento de producto a otro mejorado. Queremos el software que mejor se adapte a nuestras necesidades, pero eso es más complicado (Pressman, 2010).

En cierto modo, el proceso unificado es una forma de conseguir mejor funcionalidad y características de los modelos de procesos de software tradicionales. El proceso unificado entiende la importancia de comunicarse con los clientes y dirigir los métodos para describir su visión de un sistema (caso de uso), lo que da una importancia de la ingeniería de software y "ayuda a los arquitectos a enfocarse en los objetivos correctos, como la facilidad de comprensión, la posibilidad de cambios futuros y la reutilización", [Jac99]: recomienda un proceso iterativo e incremental que brinda un sentido evolutivo esencial en el desarrollo de software moderno (Pressman, 2010).

2.4.1. Fases del proceso unificado

Al inicio de este capítulo se estudiaron cinco actividades estructurales generales de construcción y se dice que pueden utilizarse para describir cualquier modelo de proceso de software. El proceso unificado no es la excepción, La figura 11 muestra las “etapas” del PU y las articula con las actividades generales discutidas al comienzo de este capítulo (Pressman, 2010).

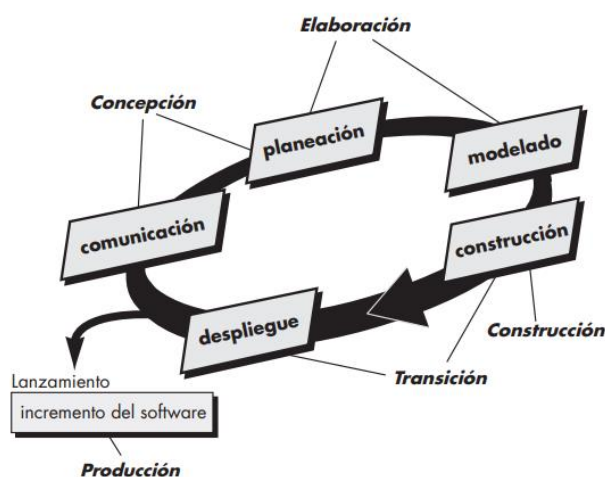


Figura 11 Proceso Unificado

Fuente: (Pressman, 2010)

La **fase de concepción** de PU combina las actividades de comunicación con el cliente y planificación, trabaja con las partes interesadas, identifica las necesidades comerciales, se plantea una arquitectura de sistema aproximada y se desarrolla un plan para la naturaleza iterativa e incremental del proyecto en curso. Los requisitos básicos para el trabajo se describen mediante un conjunto inicial de casos de uso preliminares, con detalles de las características y funcionalidades requeridas para cada categoría principal de usuarios (Pressman, 2010).

En esta fase es que la arquitectura no es más que una descripción general de los principales subsistemas y las funciones y características que tienen. La arquitectura se mejorará y ampliará en un grupo de modelos que mostrarán

diferentes vistas del sistema. La planeación identifica los recursos, valora riesgos clave, puntualiza planes de acción y establecer una línea de base para las etapas que se irán aplicando a medida que avance el proceso de incremento de los programas(Pressman, 2010).

La **fase de elaboración** integra actividades de comunicación y modelado del modelo general del proceso (ver Figura 11). Cree y amplíe los casos de uso iniciales avanzados desarrollados como parte de la fase de diseño y amplíe la representación de la arquitectura para incluir cinco vistas diferentes del programa: los modelos del caso de uso, de requerimientos, del diseño, de la implementación y del despliegue. En algunos casos, el diseño crea una "línea de base de la arquitectura ejecutable" [Arl02] que representa un sistema ejecutable de "primer corte". La línea base de la arquitectura representan la viabilidad arquitectónica, pero no provee todas las características y funciones necesarias para utilizar el sistema. Además, al final de la fase de desarrollo, el plan se revisa cuidadosamente para asegurar que el alcance, los riesgos y los plazos de entrega siguen siendo razonables. Los cambios en el plan generalmente se realizan en este punto(Pressman, 2010).

La **fase de construcción** de PU es similar a la actividad de construcción específica del proceso general del programa. Utilizando el modelo arquitectónico como entrada, la fase de construcción desarrolla o adquiere componentes de software que causará que todos los casos de uso sean operativos para el usuario final. Para ello se completan los modelos de requerimientos y diseños que se inician en la etapa de elaboración para que se reflejen laa versión final del incremento del software. Luego, todas las funciones se implementan en el código fuente, con las características y funciones necesarias para aumentar el software, se realizan pruebas unitarias para cada uno a medida que los componentes se implementan, se diseñan. También, se realizan procesos de integración (ensamblaje de componentes y pruebas de integración). Los casos de uso se utilizan para derivar un conjunto de pruebas, que se realiza antes del inicio de la siguiente etapa de la PU(Pressman, 2010).

La **fase de transición** de UP incluye las etapas finales de la actividad de construcción y la primera parte de la publicación general (entrega y feedback). Se facilita el Software a los usuarios finales para las pruebas beta, ya sea para informes de errores o cambios necesarios, aquí el equipo de software crea la información de apoyo necesaria (como manuales de usuario, guías de resolución de problemas, procedimientos de resolución de problemas, instalación, ajustes, entre otros) necesarios para el inicio. Al final del período de transición, se lanza el software que se ha mejorado para convertirlo en un producto utilizable (Pressman, 2010).

La **fase de producción** del PU concuerda con la actividad de despliegue del proceso general. En esta etapa se controla el uso del software y se brinda soporte para solicitudes de operación (infraestructura), defectos y cambios para su evaluación. Es posible que además de las fases de construcción, transición y producción, se empiece a trabajar en el próximo incremento del programa. Esto significa que las cinco fases de PU no suceden secuencialmente sino de manera escalonada. Los flujos de trabajo de ingeniería de software se distribuyen en todas las fases del PU. En su contexto, un flujo de trabajo es como un conjunto de tareas, en otras palabras, el flujo de trabajo define las tareas que se requieren para completar una acción importante de la ingeniería de software los productos de trabajo después de su éxito. Cabe señalar que no todas las tareas específicas de un flujo de trabajo de PU se implementan en todos los proyectos de software. El equipo ajusta el proceso (acciones, tareas, subtareas y productos del trabajo) para cumplir la demanda (Pressman, 2010).

2.5. Modelo proceso personal y equipo

El mejor proceso de software es aquel que está cerca de quienes harán el trabajo. Si un modelo de proceso de software es desarrollado a nivel empresarial u organizacional, solo funcionará si acepta una adaptación sustancial a las necesidades del equipo de trabajo de proyecto de ingeniería de software. En la situación ideal, crear un proceso que mejor se adapte a los requisitos y al mismo tiempo cubra las más amplias necesidades de grupos y organizaciones. El equipo creará un proceso propio que satisfaga las

necesidades concretas de las personas y las necesidades más amplias de la organización(Pressman, 2010).

2.5.1.PPS Proceso personal del software

Cada desarrollador emplea un proceso para crear programas de cómputo. El proceso puede raro o extraño, tal vez cambia todos los días, puede ser ineficaz, eficaz o incluso innecesario; pero hay un "proceso". Watts Humphrey [Hum97] propone cambiar el proceso empleado de bajo rendimiento, el individuo debe pasar las cuatro etapas, cada una de las cuales requiere entrenamiento e instrumentación meticoloso. El proceso personal del software (PPS) confirma en la medida individual tanto de la producción como de la calidad del trabajo producido. Además, el PPS requiere que un experto sea responsable de la planificación del proyecto (por ejemplo: estimar y programar actividades) y delegar el control al profesional para garantizar la calidad de todos los productos de software desarrollados. modelo PPS(Pressman, 2010).

El modelo del PPS define cinco actividades estructurales:

2.5.1.1. Planeación

Esta actividad separa los requerimientos y hace estimaciones tanto para tamaño y recursos. Además, realiza la estimación de defectos (el número de defectos proyectados para el trabajo). Todas las medidas se registran en la hoja o plantilla, finalmente, se definen las tareas de desarrollo y se crea un programa para el proyecto(Pressman, 2010).

2.5.1.2. Diseño de alto nivel

Se desarrollan especificaciones externas para cada componente construido y se crea el diseño del componente. Si hay sospecha, se hacen prototipos. Todos los aspectos relevantes son registrados y rastreados(Pressman, 2010).

2.5.1.3. Revisión del diseño de alto nivel

Se aplican métodos oficiales de verificación para revelar fallas de diseño. Se mantienen las medidas para todas las tareas importantes y resultados del trabajo(Pressman, 2010).

2.5.1.4. Desarrollo

Diseño y revisión de componentes mejorados. Código generado, revisado, compilado y probado. Los indicadores se mantienen para todas las tareas importantes y los resultados del trabajo(Pressman, 2010).

2.5.1.5. Post Mortem

La eficiencia del proceso está determinada por medidas y mediciones obtenidas (esta es una gran cantidad de datos que deben ser analizados usando métodos estadísticos). Las medidas y mediciones deben proporcionar orientación para modificar el proceso para mejorar su eficacia(Pressman, 2010).

PPS enfatiza la necesidad de una detección temprana de errores; es igualmente importante entender los tipos con el que es probable cometer. Esto se logra a través de la actividad de evaluación, aplicada a todos los productos de trabajo que sean creados. PPS incorpora un enfoque disciplinado y fundamentado en la medición de la ingeniería de software que puede resultar un choque cultural para muchos de los que lo practican. Aunque, cuando PPS se ha presentado correctamente a los ingenieros de software [Hum96], es importante que el resultado sea una mejora de la productividad técnica y la calidad del software [Fer9 97], sin embargo, PPS no es ampliamente adoptado por la industria. Desafortunadamente, la razón de esto tiene más que ver con la naturaleza humana y las deficiencias organizacionales más no de las fortalezas y debilidades del enfoque PPS. Tal enfoque plantea desafíos intelectuales y requiere un nivel de compromiso (de los profesionales y sus administradores) que no siempre se puede obtener. El proceso de formación es relativamente largo y caro. El nivel requerido de medición es culturalmente difícil para muchas personas de la comunidad de software(Pressman, 2010).

2.5.2. Proceso del equipo de software (PES)

Dado que muchos proyectos de software industrial están realizados por un equipo de expertos, Watts Humphrey amplió las lecciones aprendidas de la introducción de PPS y recomendó un proceso de equipo de software (PES) con la finalidad de formar un equipo de proyecto "auto dirigido", que se organice para producir software de alta calidad. Humphrey [Hum98] establece los siguientes objetivos para PES: (Pressman, 2010)

- Formar equipos auto dirigidos para planificar y monitorear su trabajo, establecer metas y hacerse cargo de sus procesos y planes. Esto podría ser programa puro o producto integrado (EPI) conformados de 3 a 20 ingenieros.
- Mostrar a los gerentes cómo liderar y motivar a sus equipos y cómo apoyarlos mantener un rendimiento óptimo.
- Aligerar el proceso de mejora del software, proporcionando un modelo de madurez de la capacidad, CMM, nivel 5, comportamiento normal y esperado.
- Proporcionar a las organizaciones maduras evidencia de mejora.
- Proporcionar la enseñanza de habilidades grupales a nivel sectorial a nivel universitario.

El equipo auto dirigido tiene una percepción consistente de sus metas y objetivos generales; precisa los roles y responsabilidades de cada miembro del equipo; proporcionar seguimiento cuantitativo a los datos del proyecto (en términos de productividad y calidad); reconoce el proceso de grupo idóneo para el proyecto y una estrategia para su implementación; Determina estándares locales para el trabajo del equipo de ingeniería de software; valora continuamente los riesgos y responde adecuadamente; supervisa, gestiona e informa sobre el estado del proyecto (Pressman, 2010).

PSE detalla las siguientes actividades estructurales: **inicio del proyecto, diseño de alto nivel, implementación, integración y pruebas, y post mórtem**. Como sus contrapartes en PPS (tenga en cuenta que los términos son ligeramente diferentes), estas actividades permiten que el grupo

planifique, diseñe y construya software de manera disciplinada, y que mida cuantificación de procesos y los productos. El período post-mortem es el escenario de mejoras proceso. PES utiliza una diversidad de scripts, formatos y estándares para guiar a sus miembros del equipo de trabajo; scripts que definen operaciones para un proceso (por ejemplo: Proyecto de puesta en marcha, diseño, implementación, integración de sistemas, pruebas y post-mortem), además de otras funciones más especificadas del trabajo (planeación del desarrollo, desarrollo de requerimientos, administración de la configuración del software y prueba unitaria) que son parte del proceso de grupo(Pressman, 2010).

PES concibe que los mejores equipos de software son auto dirigidos. Los miembros del equipo determinan las metas del proyecto, organiza el proceso para que satisfaga las necesidades y controlan la planificación de las actividades del proyecto, con medición y análisis de las tomas realizadas, se trabaja continuamente para mejorar el enfoque de ingeniería de software disponible. Tanto PPS, PES son un enfoque riguroso para la ingeniería de software y brinda beneficios distintivos y cuantificables en términos de productividad y calidad. El equipo debe estar íntegramente comprometido con el proceso y completamente capacitado para garantizar que se siga el enfoque adecuadamente(Pressman, 2010).

Referencia Bibliográfica

- Aguilar Vera, R. A., Oktaba, H., Juárez-Ramírez, R., Aguilar Cisneros, J. R., Fernández-y-Fernández, C. A., Rodríguez Elias, O. M., & Ucán Pech, J. P. (2017). Ingeniería de Software. In *La Computación en México por especialidades académicas*.
- Braude, E. (2001). El Proceso de Software La Ingeniería del Software. *Software Engineering*. http://ocw.uc3m.es/ingenieria-informatica/disenio-de-software-avanzado/material-de-clase-1/01-El_Proceso_de_Desarrollo_de_Software.pdf
- Ciclo de vida del software: todo lo que necesitas saber*. (2020). <https://intelequia.com/blog/post/2083/ciclo-de-vida-del-software-todo-lo-que-necesitas-saber>
- García, F. J., Moreno, M. N., García, A., & Vázquez, A. (2019). *Ingeniería de Software I*. 23. [https://repositorio.grial.eu/bitstream/grial/1940/1/IS_I Tema 3 - Modelos de Proceso.pdf](https://repositorio.grial.eu/bitstream/grial/1940/1/IS_I_Tema_3_-_Modelos_de_Proceso.pdf)
- ISO. (2017). *ISO/IEC/IEEE 12207:2017(en), Systems and software engineering — Software life cycle processes*. <https://www.iso.org/obp/ui/#iso:std:iso-iec-ieee:12207:ed-1:v1:en>
- JAN Sommerville. (2005). *Ingeniería del Software*. <https://www.google.com.mx/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&ved=2ahUKEwjql-Sev6jtAhUlrZ4KHajDA3UQFjACegQIBBAC&url=https%3A%2F%2Femtinforada.files.wordpress.com%2F2015%2F03%2Fingenierc3ada-del-software-ian-sommerville.pdf&usg=AOvVaw1flqBuDj35mOw1qOlc0>
- Pickin, S., & Valls, M. G. (2006). Introducción a la Ingeniería del Software. In *Chaos*.
- Pressman, R. S. (2010). *Ingeniería del Software Un enfoque práctico* (Séptima Ed). McGraw-Hill. <http://cotana.informatica.edu.bo/downloads/Id-Ingenieria.de.software.enfoque.practico.7ed.Pressman.PDF>